Journal of Computational and Applied Mathematics 414 (2022) 114418

Contents lists available at ScienceDirect

Journal of Computational and Applied Mathematics

journal homepage: www.elsevier.com/locate/cam

When and how to split? A comparison of two IMEX splitting techniques for solving advection–diffusion–reaction equations

Adam Preuss [a], Jessica Lipoth [b], Raymond J. Spiteri [a,*]

[a] Department of Computer Science, College of Arts and Science, University of Saskatchewan, 176 Thorvaldson Building, 110 Science Place, Canada

[b] Department of Electrical and Computer Engineering, College of Engineering, University of Saskatchewan, 57 Campus Dr, Canada

ARTICLE INFO

Article history:
Received 3 February 2022
Received in revised form 7 May 2022

MSC:
65L04
65L05
65L06
65M20
65Y20

Keywords:
Additive Runge–Kutta methods
Advection–reaction–diffusion equations
Time integration
Splitting methods

ABSTRACT

Many mathematical models of natural phenomena are described by partial differential equations (PDEs) that consist of additive contributions from different physical processes. Classical methods for the numerical solution to such equations are monolithic in that all processes are treated with a single method. Additive numerical methods, in contrast, apply distinct methods to each additive term. There are, however, different ways mathematically to specify the additive terms, and it is not always clear which ways (if any) offer advantages over monolithic methods. This study compares the performance of two different additive splitting techniques (physics-based splitting and dynamic linearization) on a suite of eight test problems that involve advection, reaction, and diffusion with various 2-additive Runge–Kutta methods and (monolithic) Runge–Kutta–Chebyshev (RKC) methods. Results show that dynamic linearization generally outperforms physics-based splitting and so should be preferred as the splitting technique when splitting is required or otherwise desirable. RKC methods are the best performers on three of the eight problems, especially at coarse tolerances, but they can also be prone to severe underperformance.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

The evolution of many physical systems can be modelled mathematically through the use of partial differential equations (PDEs). After spatial discretization, these problems take the form of an initial-value problem (IVP) for a system of ordinary differential equations where the solution is completely specified at some initial state. An IVP for an ordinary differential equation (ODE) can be defined in the form

$$\frac{d\mathbf{y}}{dt}(t) = \mathbf{f}(t, \mathbf{y}(t)), \quad t \in [t_0, t_{\text{final}}], \quad \mathbf{y}(t_0) = \mathbf{y}_0, \tag{1}$$

where $t \in \mathbb{R}$ is time, $\mathbf{y}(t) \in \mathbb{R}^m$ is the solution to the IVP at t, $\mathbf{f} : \mathbb{R} \times \mathbb{R}^m \mapsto \mathbb{R}^m$ is a function defining the right-hand side (RHS) of the ODE, t_0, t_{final} are the initial and final times, respectively, and $\mathbf{y}_0$ is the initial condition.

Often, the RHS of an ODE is naturally comprised of a sum of contributing factors. In contrast to classical methods for the numerical solution of Eq. (1) that treat the RHS of such problems monolithically, additive Runge–Kutta (ARK)

* Corresponding author.
E-mail address: spiteri@cs.usask.ca (R.J. Spiteri).

https://doi.org/10.1016/j.cam.2022.114418
0377-0427/© 2022 Elsevier B.V. All rights reserved.

methods apply distinct methods to each contributing factor in the hopes of achieving better performance [1–4]. A similar treatment could be achieved by using linear multi-step methods or indeed general linear methods [5], but because this study focuses on splitting techniques, e.g., [6] and references therein, we consider only ARK methods as time integrators. Linear multistep methods are also not considered in this study because the theoretical stability properties of RK methods are better for higher orders [7].

The most popular type of ARK method is the (2-additive) *implicit–explicit* (IMEX) method. In this case, Eq. (1) is written as

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}^{\{\mathrm{I}\}}(t, \mathbf{y}(t)) + \mathbf{f}^{\{\mathrm{E}\}}(t, \mathbf{y}(t)), \quad \mathbf{y}(0) = \mathbf{y}_0, \tag{2}$$

where $t \in \mathbb{R}$, $\mathbf{y} \in \mathbb{R}^m$, and $\mathbf{f}^{\{\mathrm{I}\}}, \mathbf{f}^{\{\mathrm{E}\}} : \mathbb{R} \times \mathbb{R}^m \mapsto \mathbb{R}^m$. An implicit numerical method is applied to $\mathbf{f}^{\{\mathrm{I}\}}$, whereas an explicit method is applied to $\mathbf{f}^{\{\mathrm{E}\}}$. It is quite common for the RHS of an ODE to contain terms with varying contributions to stiffness [3], and so the idea is to capture the stiffness via the $\mathbf{f}^{\{\mathrm{I}\}}$ term and treat it with an implicit method to ameliorate step-size restrictions for stability. The $\mathbf{f}^{\{\mathrm{E}\}}$ term is assumed to capture all the non-stiff contributions from the RHS and is treated with an explicit method. Treating the stiff and non-stiff terms separately often allows for better performance [7] compared to both implicit or explicit monolithic methods: the decrease in stability compared to fully implicit methods is more than offset by the reduction in cost per step, and conversely, the increase in stability compared to fully explicit methods more than offsets the increase in cost per step. Moreover, specialized methods can often be applied to the sub-problems. We note that it is also possible perform a more specialized splitting within the ARK framework, so-called *fractional-step methods*, where only one operator of the ODE is advanced at a time [8]. A systematic comparison of the performance IMEX and fractional-step methods, although interesting and perhaps overdue, is beyond the scope of this study and left as a potential topic for future work.

An important issue affecting the performance of IMEX methods is how exactly to split the RHS of Eq. (1) into the form Eq. (2). A common form of 2-additive splitting is *physics-based* splitting. In physics-based splitting, the two additively split terms represent some physically significant contributing factors. It is often the case that there is a strong correlation between physical processes and their stiffness. Examples include splitting stiff acoustic wave propagation terms from non-stiff windspeed terms in the HOMME-NH dynamical core [9], splitting stiff vertical waves from non-stiff horizontal waves in atmospheric models [10], and splitting stiff diffusion terms from non-stiff convections terms in sedimentation models [11].

A second form of splitting, which does not typically reflect any physical characteristics of an ODE, is *dynamic linearization*. In dynamic linearization, the RHS of Eq. (1) is linearized about the solution to obtain

$$\frac{d\mathbf{y}}{dt} = \mathbf{J}_{\mathbf{f}}(t, \mathbf{y})\mathbf{y} + [\mathbf{f}(t, \mathbf{y}) - \mathbf{J}_{\mathbf{f}}(t, \mathbf{y})\mathbf{y}], \tag{3}$$

where $\mathbf{J}_{\mathbf{f}}(t, \mathbf{y})$ is the Jacobian of $\mathbf{f}$ evaluated at t and $\mathbf{y}(t)$. In practice, the first term on the RHS of Eq. (3) is frozen at the beginning of a time step, leading to a linear, constant-coefficient $\mathbf{f}^{\{\mathrm{I}\}}$ term in Eq. (2). Moreover, the Jacobian of the term corresponding to $\mathbf{f}^{\{\mathrm{E}\}}$ on the right-hand side of Eq. (3) satisfies $\mathbf{J}_{\mathbf{f}^{\{\mathrm{E}\}}}(\mathbf{y}_n) = \mathbf{0}$ [12], implying that the Lipschitz constant of $\mathbf{f}^{\{\mathrm{E}\}}$ is (much) smaller than that of $\mathbf{f}$ and justifying its treatment with an explicit method. Dynamic linearization is thus a splitting based on numerical properties of the solution and generalizes the popular and effective linear-nonlinear/stiff-nonstiff splitting techniques.

Although not always referred to as such, dynamic linearization in the context of additive numerical methods has been around since the 1960s [13]; see also [1,2]. An example of dynamic linearization combined with Rosenbrock methods is [14], in which the Rosenbrock methods are able to solve a parabolic problem and a 2D advection–diffusion–reaction (ADR) problem on par or more efficiently than explicit methods. More recently, dynamic linearization combined with a new class of exponential Rosenbrock methods has been used in [12] to solve systems pertaining to animation. Using this technique, an increase is achieved in efficiency and accuracy over classical methods for a stiff-spring model; correct-looking animations of a car crash and elastic deformations, among others, are also produced. An example of dynamic linearization combined with exponential methods is given in [15]; this technique is used to solve a multi-layer ocean model with greater efficiency than explicit methods.

This paper compares the performance of physics-based splitting and dynamic linearization on a suite of eight test problems with emphasis on ADR PDEs, although the models for some problems only have two of the three physical processes present. Typically, physics-based splitting is applied to ADR PDEs [16] because the problem is naturally split into advection, diffusion, and reaction terms. However, the typical IMEX approach requires a decision as to how these three contributions are aggregated into two, one stiff and one non-stiff. Such a decision cannot always be made in an immutable fashion with success for an entire problem because the relative stiffness of the terms may change during the simulation; e.g., reactions may become relatively stiff or non-stiff relative to diffusion over time. Dynamic linearization may be able to adaptively group numerically significant characteristics that are not represented as physical processes.

Both physics-based splitting and dynamic linearization are applied to the test suite by using 2-additive IMEX Runge–Kutta (RK) methods of orders three through five. The performance of the two splitting methods is evaluated by comparing the run times and accuracies of the methods for each problem. To provide a baseline for comparison to standard (monolithic) numerical methods used in practice, second-order Runge–Kutta–Chebyshev (RKC) methods are also applied to the problems. We find that RKC methods occasionally outperform the IMEX methods in terms of speed for low-accuracy

solutions. We chose RKC methods over the fourth-order ROCK4 methods because of the availability of a mature and robust RKC solver. The use of second-order RKC methods also provides data on the tradeoff between low- and high-order methods in practice.

The rest of this paper is organized as follows. Section 2 outlines the numerical methods studied and the technique used to compare the numerical integrators and splitting methods. Section 3 provides a definition of each problem in the test suite. Section 4 discusses the relevant results regarding the two types of splitting on ADR equations. Section 5 concludes the paper by summarizing the relevant results and providing insight into the implications of this work.

2. Numerical methods

This study considers five numerical methods: IMEX ARK methods of orders three, four, and five, as well as RKC methods of orders one and two. Other methods such as the Dormand–Prince explicit embedded Runge–Kutta pair [17], the IMEX-RKC method IRKC [18], the partitioned RKC method PRKC [19], and DIRK [20] were also considered but ultimately excluded from the study because they are found to consistently underperform relative to the included methods. Higher-order IMEX methods are used with the intent of solving the ADR equations both efficiently and accurately. All the numerical methods tested were run with variable step sizes.

2.1. IMEX-RK methods

The general form of an IMEX-RK method is given by

$$\begin{aligned}
\mathbf{k}_i &= \mathbf{f}^{\{\mathrm{I}\}}\left(t_{n-1}+c_i\Delta t_n, \mathbf{y}_{n-1}+\Delta t_n\sum_{j=1}^{s}\left[a_{ij}\mathbf{k}_j+\tilde{a}_{ij}\tilde{\mathbf{k}}_j\right]\right),\\
\tilde{\mathbf{k}}_i &= \mathbf{f}^{\{\mathrm{E}\}}\left(t_{n-1}+\tilde{c}_i\Delta t_n, \mathbf{y}_{n-1}+\Delta t_n\sum_{j=1}^{s}\left[a_{ij}\mathbf{k}_j+\tilde{a}_{ij}\tilde{\mathbf{k}}_j\right]\right), \quad i=1,2,\ldots,s,\\
\mathbf{y}_n &= \mathbf{y}_{n-1}+\Delta t_n\sum_{i=1}^{s}\left[b_i\mathbf{k}_i+\tilde{b}_i\tilde{\mathbf{k}}_i\right],
\end{aligned}$$

where $\mathbf{f}^{\{\mathrm{I}\}}$ denotes the stiff part of the RHS and $\mathbf{f}^{\{\mathrm{E}\}}$ denotes the non-stiff part. An implicit RK method with coefficients a_{ij}, b_i, c_i is used on the stiff part and an explicit RK (ERK) method with coefficients $\tilde{a}_{ij}, \tilde{b}_i, \tilde{c}_i$ is used on the non-stiff part.

The third-order IMEX ARK method used within this study is the method ARK3(2)4L[2]SA-ESDIRK, taken from [4]. This method was chosen because it was developed for similar convection–diffusion–reaction equations. For simplicity, it is referred to here as ARK3(2).

Numerous fourth-order IMEX ARK methods have been developed over the past decade. A comprehensive overview of the methods, as well as their strengths and weaknesses, is given in [4]. The method selected for this study is the method ARK4(3)6L[2]SA-ESDIRK; this method is also from [3] and was chosen because of its development for similar convection–diffusion–reaction equations. For simplicity, it is referred to here as ARK4(3).

The fifth-order IMEX ARK method selected for this study is the method ARK5(4)8L[2]SA-ESDIRK from [3]. For simplicity, it is referred to here as ARK5(4). To the best of the authors' knowledge, the only known ARK5(4) methods are given in [3,4]; the ARK5(4) method from [3] is selected because it has been crafted for similar convection–diffusion–reaction equations. Unlike the ARK4(3) method, the ARK5(4) method has known problems with stiffness leakage for stiff problems [3].

2.2. RKC methods

RKC methods are a class of ERK methods that aim to overcome the stability constraints of general ERK methods by using an extended region of absolute stability along the negative real axis, making them particularly suitable for ODEs derived from spatial discretizations of parabolic PDEs. The absolute stability region $R_s(z)$ for an s-stage RKC method grows as s^2, generally providing a favourable tradeoff of increase in stable step size (while maintaining a sufficiently accurate solution) versus cost per step. The maximization of the absolute stability region leads to the use of Chebyshev polynomials in the definition of the stages. The recursive definition of the Chebyshev polynomials allows the stages to also be defined recursively; thus, implementation of an RKC method has minimal storage requirements.

The number of stages used for a given timestep is adapted based on the spectral radius Λ of $\mathbf{J}_{\mathbf{f}}(t_{n-1}, \mathbf{y}_{n-1})$, i.e., the eigenvalue of the Jacobian of the RHS evaluated at the beginning of the timestep with the largest absolute value. The calculation for the number of stages for each step is written [21]

$$s = 1+\left\lfloor\sqrt{1+\frac{\Lambda\Delta t_n}{0.653}}\right\rfloor$$

Table 1
Test problems and method used to generate reference solutions.

Test problem	Reference solution solver
Linear advection–diffusion	RODAS
2D Heat Transfer	RODAS
1D Brusselator	RODAS
2D Brusselator	RADAU5
CUSP	RODAS
Combustion	RADAU5
Tumour Angiogenesis	RADAU5
Concrete-Rewetting	RADAU5

to guarantee that the stability condition $\Lambda\Delta t_n \leq \beta(s)$ is satisfied, where $\beta(s) = \max\{-z : z \leq 0, |R_s(z)| \leq 1\}$ is the real stability boundary. The s-stage formula for an RKC method is written as [21]

$$\begin{aligned}
\mathbf{y}_0 &= \mathbf{y}_{n-1},\\
\mathbf{y}_1 &= \mathbf{y}_0 + \tilde{\mu}_1 \Delta t_n \mathbf{f}_0,\\
\mathbf{y}_j &= (1-\mu_j-\nu_j)\mathbf{y}_0 + \mu_j \mathbf{y}_{j-1} + \nu_j \mathbf{y}_{j-2} + \tilde{\mu}_j \Delta t_n \mathbf{f}_{j-1} + \tilde{\gamma}_j \Delta t_n \mathbf{f}_0,\\
&\qquad j = 2, 3, \ldots, s,\\
\mathbf{y}_n &= \mathbf{y}_s,
\end{aligned}$$

where

$$\mathbf{f}_j = \mathbf{f}(t_{n-1} + c_j \Delta t_n, \mathbf{y}_j),$$

the coefficients $\tilde{\mu}_j$, μ_j, ν_j, γ_j are specified according to [21]. The damping parameter is also specified according to [21] for the first- and second-order cases, respectively, leading in each case to about a 2% reduction in $\beta(s)$ while regularizing the pinch points in the stability region on the negative real axis.

2.3. Procedure for running experiments

This study utilizes `pythODE++`, a specialized problem-solving environment (PSE) developed specifically to compare numerical methods and splitting techniques for IVPs in ODEs [22]. The `pythODE++` PSE allows the creation and analysis of large numerical experiments containing multiple permutations of numerical methods, tolerances, and IVPs. Within an experiment, `pythODE++` iterates over the specified parameter sets and then provides an analysis of the generated experimental data. The sets of parameters within an experiment and the statistics generated after are flexible and determined by the user.

The `pythODE++` PSE is based on `pythODE` [23], a similar PSE written in the Python programming language. However, `pythODE++` is more specialized and focuses on efficiency; as such, `pythODE++` is written primarily in C++ because it is a fast and powerful language [24]. The `pythODE++` PSE also utilizes OpenMPI to allow individual parameter sets within experiments to run simultaneously, thus decreasing the overall runtime of the experiments. The use of other external libraries within `pythODE++` is minimal to avoid unnecessary overhead and to ensure control over the timing of experiments.

Numerical methods implemented within `pythODE++` are verified on a suite of thirty stiff and thirty non-stiff IVPs having known exact solutions, as well as the HIRES, Beam, and van der Pol problems from the Bari test set [25]. The full suite of thirty stiff and non-stiff problems can be found in [26]; more details on `pythODE++` can be found in [22].

In this study, each IVP in the test suite of problems (see Section 3) is run as an individual experiment, iterating over the specified numerical methods and specified tolerances for that IVP. These experiments result in the generation of work-precision diagrams, which are then used to compare the performance of the various splitting techniques on the IMEX and RKC methods in Section 4.

In order to evaluate a numerical method's accuracy in the absence of closed-form exact solutions, a highly accurate reference solution is generated for each experiment as a proxy. For this paper, we generate reference solutions by solving the IVPs with either the RODAS or RADAU5 variable-stepsize codes because they are known to perform well on many IVPs [27]. Absolute and relative tolerances were set to produce solutions that were one or two orders of magnitude below those observed in the experiments. Generally, RODAS is preferred because it is faster; however, there are a few problems for which RAUDAU5 performs better. Table 1 shows the test suite of problems and the method used to generate the reference solution. The problems for which a published solution is available, as well as the reference containing the solution, are listed in Table 2.

The performance of a given method in this study depends critically on the computation, storage, and inversion of the Jacobian matrices. Jacobians are computed either analytically or using finite differences; analytical computation can be more efficient depending on the size and density of the Jacobian. In any case, ADOL-C [30] was used to validate the

Table 2
Numerical studies with published solutions for comparison.

Test problem	Published solution
1D Brusselator	[27]
2D Brusselator	[27]
CUSP	[27]
Combustion	[28]
Tumour Angiogenesis	[16]
Concrete Rewetting	[29]

implementation. The Jacobians were then stored in a format compatible for use with the SuiteSparse package of sparse matrix algorithms [31]. The ensuing linear systems were solved using the direct sparse `umfpack_zi_solve` solver [32].

Finally, all problems are run using a machine with the following specifications:

- 2x Xeon E5-2620 (6C 2.0 GHz 15 MB 1333 MHz 95 W)
- 4x 8 GB RAM (16 GB per processor)
- 500 GB RAM
- Gigabit Ethernet
- Ubuntu 16.04 LTS

The machine is capable of executing 24 simultaneous threads (2 processors per machine $\times$ 6 cores per processor $\times$ 2 threads per core). However, problems are run only with 12 threads to avoid timing issues caused by overloading the processes.

3. Test suite of problems

We now give brief mathematical descriptions of the eight problems that comprise the test suite for this study. In this section, we specify the differential equations, their boundary and initial conditions, their spatial discretizations, and parameter values used. In each case, the two splittings considered are physics-based splitting and dynamic linearization. Detailed expressions for $\mathbf{f}^{\{\mathrm{I}\}}$ and $\mathbf{f}^{\{\mathrm{E}\}}$ are generally omitted for the sake of brevity; they can be found in [22]. The numerical results from applying the various methods considered are reported in Section 4.

3.1. 1D advection-diffusion

The first numerical study consists of a basic one-dimensional, constant-coefficient advection–diffusion equation for a scalar quantity q, given by

$$\begin{aligned} &\frac{\partial q}{\partial t} + a\frac{\partial q}{\partial x} = d\frac{\partial^2 q}{\partial x^2}, \\ &\quad q(0,x) = \sin(2\pi x), \\ &\quad q(t,0) = q(t,1), \end{aligned} \tag{4}$$

where $x \in [0,1]$, and $a \in \mathbb{R}$ and $d \in \mathbb{R}$ are constant scalars representing the strength of advection and diffusion, respectively. The values used are $a = \{10, 100\}$ and $d = \{10, 100\}$.

The spatial domain of the equation is uniformly discretized into N points with spacing $\Delta x = 1/N$; a second-order centred finite difference method yields a system of ODEs written as

$$\frac{dq_i}{dt} = d\left(\frac{q_{i+1} - 2q_i + q_{i-1}}{(\Delta x)^2}\right) - a\left(\frac{q_{i+1} - q_{i-1}}{\Delta x}\right), \tag{5}$$

where $i = 1, 2, \ldots, N$. Although an upwind difference for the advection term could have been used for this problem, a centred difference provides for a more generic (less specialized) approach. The periodic boundary conditions are imposed as $q_0(t) = q_N(t)$ and $q_{N+1}(t) = q_1(t)$.

To apply physics-based splitting to the problem, the diffusion term is treated implicitly, and the advection term is treated explicitly.

We note that this problem is linear with constant coefficients; as such, dynamic linearization reduces to a fully implicit method

$$\mathbf{f}^{\{\mathrm{I}\}} = (\mathbf{D} - \mathbf{A})\mathbf{q}, \qquad \mathbf{f}^{\{\mathrm{E}\}} = \mathbf{0},$$

where vector $\mathbf{q} = (q_1, q_2, \ldots, q_N)^T$ and matrices $\mathbf{D} = d/(\Delta x)^2\,\mathrm{diag}(1, -2, 1)$ and $\mathbf{A} = a/\Delta x\,\mathrm{diag}(1, 0, -1)$.

This simple example is used to demonstrate how physics-based splitting performs poorly compared to dynamic linearization when the stiffness of the advection term increases; see Section 4.1.

3.2. 2D heat transfer

A model of heat transfer in an infinite three-dimensional prismatic channel with a known velocity profile is presented in [33]. The model considered here is a two-dimensional linear advection–diffusion PDE for a horizontal slice of the channel. The interior of the slice is initially at a constant, relatively cool temperature. The inlet temperature to the slice is hot, and the conductance of the side walls is prescribed.

The slice is defined to be of length l and width h. It is aligned lengthwise in the x-direction and widthwise in the y-direction; thus, the domain can be taken to be $x \in [0, l]$ and $y \in [-h/2, h/2]$. The PDE, boundary conditions, and initial condition of the model for temperature T are written

$$\begin{aligned} \frac{\partial T}{\partial t} &= d\,\nabla^2 T - \mathbf{u}\cdot\nabla T, \\ \frac{\partial T}{\partial y}(t, x, -h/2) &= -\frac{\partial T}{\partial y}(t, x, h/2) = \sigma, \\ T(t, 0, y) &= T_{\text{in}}, \qquad T(0, x, y) = T_0, \end{aligned} \tag{6}$$

where d is the diffusivity coefficient, $\mathbf{u}$ is the velocity field in the channel, T_{in} is the input temperature, T_0 is the initial temperature of the channel, and σ is a measure of the heat flux across the channel wall, and a free boundary condition is taken at the outlet $x = l$.

The fluid is taken to travel lengthwise down the channel, with the velocity $\mathbf{u}$ taken to have a parabolic profile given by

$$\mathbf{u}(x, y) = \begin{pmatrix} \frac{3}{2}u_{\text{avg}}\left[1 - \left(\frac{y}{h/2}\right)^2\right] \\ 0 \end{pmatrix},$$

where u_{avg} is the average velocity. The problem parameters are chosen as $h = 0.1$ m, $l = 0.7$ m, $d = 1.38 \times 10^{-7}$ m^2/s, $u_{\text{avg}} = 4.5 \times 10^{-6}$ m/s, $\sigma = 0$ K/m, $T_0 = 298.15$ K, and $T_{\text{in}} = 303.15$ K.

This PDE is uniformly discretized on a two-dimensional grid, where the unknown grid points consist of N_x points in the x-direction and N_y points in the y-direction at the cell centres. Applying second-order centred differences to the second derivatives and first-order backward differences to the first derivatives yields a discretized system that is written as

$$\begin{aligned} \frac{dT_{ij}}{dt} &= d\left(\frac{T_{i-1,j} - 2T_{i,j} + T_{i+1,j}}{(\Delta x)^2} + \frac{T_{i,j-1} - 2T_{i,j} + T_{i,j+1}}{(\Delta y)^2}\right) \\ &\quad - \frac{3}{2}u_{\text{avg}}\left[1 - \left(\frac{(i+\frac{1}{2})\Delta y - \frac{h}{2}}{h/2}\right)^2\right]\frac{T_{i,j} - T_{i-1,j}}{\Delta x}, \end{aligned} \tag{7}$$

with boundary values written as

$$T_{0,j}(t) = T_{\text{in}}(t), \quad T_{i,0}(t) = T_{i,1}(t) - (\Delta y)\sigma, \quad T_{i,N+1}(t) = T_{i,N}(t) - (\Delta y)\sigma,$$

where $i = 1, 2, \ldots, N_x$, $j = 1, 2, \ldots, N_y$, $\Delta x = \frac{l}{N_x}$, and $\Delta y = \frac{h}{N_y}$. The outlet boundary condition is imposed as (7) but with one-sided differences in x toward the interior of the domain.

Physics-based splitting and dynamic linearization are performed the same as for the above problem in Section 3.1. The primary difference between this model and the one-dimensional advection-diffusion problem is that the Jacobian of the RHS of the heat transfer problem contains two additional bands because the PDE is two-dimensional.

3.3. 1D brusselator

The Brusselator is a theoretical model of an autocatalytic reaction consisting of two reacting quantities [16]. The model consists of two coupled diffusion–reaction equations, with linear diffusion and non-linear reaction, written as

$$\begin{aligned} \frac{\partial u}{\partial t} &= \alpha\nabla^2 u + A + u^2 v - (B+1)u, \\ \frac{\partial v}{\partial t} &= \alpha\nabla^2 v + Bu - u^2 v, \end{aligned} \tag{8}$$

where A and B are parameters governing the reaction terms and α is the diffusivity. If $B > 1 + A^2$, the solution is oscillatory; otherwise, it approaches a steady state.

The spatial domain is $x \in [0, 1]$, and accordingly ∇ simplifies to $\frac{\partial}{\partial x}$. Prescribed Dirichlet boundary and initial conditions are written as

$$\begin{aligned} u(0, t) &= u(1, t) = A, & v(0, t) &= v(1, t) = B, \\ u(x, 0) &= A + \sin(2\pi x), & v(x, 0) &= B. \end{aligned}$$

Using second-order centred differences, the uniform discretization of the one-dimensional Brusselator equation using N unknown interior grid points is written as

$$\frac{du_i}{dt} = \frac{\alpha}{(\Delta x)^2}(u_{i-1} - 2u_i + u_{i+1}) + A + u_i^2 v_i - (B+1)u_i,$$
$$\frac{dv_i}{dt} = \frac{\alpha}{(\Delta x)^2}(v_{i-1} - 2v_i + v_{i+1}) + Bu_i - u_i^2 v_i,$$

where $i = 1, 2, \ldots, N$, $\Delta x = 1/(N+1)$, with boundary and initial conditions written as

$$u_0(t) = u_{N+1}(t) = A, \qquad v_0(t) = v_{N+1}(t) = B,$$
$$u_i(0) = A + \sin(2\pi i \Delta x), \qquad v_i(0) = B.$$

This example utilizes parameters $A = 1$, $B = 3$, and $\alpha = \{1/50, 1/500\}$. These parameters lead to oscillatory behaviour in the solution.

3.4. 2D brusselator

The two-dimensional form of the Brusselator model is similar to the one-dimensional form, with an additional forcing (reaction) term $f = f(x, y, t)$ that acts on u; otherwise, the diffusion quickly dominates any of the interesting solution characteristics. The spatial domain is $(x, y) \in [0, 1]^2$. Boundary conditions are taken to be periodic; the initial conditions are given by

$$u(x, y, 0) = 22y(1-y)^{3/2}, \qquad v(x, y, 0) = 27x(1-x)^{3/2}.$$

The uniformly discretized two-dimensional Brusselator model using second-order centred differences and $N \times N$ unknown grid points is written

$$\frac{du_{i,j}}{dt} = \frac{\alpha}{\Delta^2}(u_{i,j-1} + u_{i-1,i} - 4u_{i,j} + u_{i+1,j} + u_{i,j+1}) + f_{i,j}(t) + A + u_{i,j}^2 v_{i,j} - (B+1)u_{i,j},$$
$$\frac{dv_{i,j}}{dt} = \frac{\alpha}{\Delta^2}(v_{i,j-1} + v_{i-1,i} - 4v_{i,j} + v_{i+1,j} + v_{i,j+1}) + Bu_{i,j} - u_{i,j}^2 v_{i,j},$$

where $i = 1, 2, \ldots, N$, $j = 1, 2, \ldots, N$, $\Delta = \Delta x = \Delta y = 1/(N+1)$, and the forcing term $f_{i,j}(t)$ is taken as in [27, p. 152] to be

$$f_{i,j}(t) = \begin{cases} 5 & \text{if } (i\Delta x - 0.3)^2 + (j\Delta y - 0.5)^2 \le 0.1^2 \text{ and } t \ge 1.1, \\ 0 & \text{otherwise.} \end{cases}$$

This example utilizes parameters $A = 1$, $B = 3.4$, and $\alpha = \{0.02, 0.002\}$.

3.5. CUSP

The CUSP problem is the combination of Zeeman's cusp catastrophe model and the van der Pol oscillator [16]. It is a diffusion–reaction equation that consists of three coupled PDEs. The diffusion of each PDE is linear (and uncoupled), and the reaction terms are non-linear. This particular system is of interest due to stiffness in the reaction term. The CUSP problem is defined over the domain $x \in [0, 1]$ and written as

$$\begin{aligned} \frac{\partial y}{\partial t} &= \sigma \frac{\partial^2 y}{\partial x^2} - \frac{1}{\epsilon}(y^3 + ay + b), \\ \frac{\partial a}{\partial t} &= \sigma \frac{\partial^2 a}{\partial x^2} + b + \frac{0.07(y-0.7)(y-1.3)}{(y-0.7)(y-1.3)+0.1}, \\ \frac{\partial b}{\partial t} &= \sigma \frac{\partial^2 b}{\partial x^2} + (1-a^2)b - a - 0.4y + \frac{0.035(y-0.7)(y-1.3)}{(y-0.7)(y-1.3)+0.1}, \end{aligned} \tag{9}$$

where $\sigma = 1/144$ represents the diffusivity and $\epsilon = 10^{-4}$ is a parameter controlling the stiffness of the reaction. As ϵ approaches zero, the reaction in y becomes more stiff. The PDEs are discretized on a uniform grid using N interior grid points and second-order centred differences; they are written as

$$\frac{dy_i}{dt} = \sigma\left(\frac{y_{i-1} - 2y_i + y_{i+1}}{(\Delta x)^2}\right) - \frac{1}{\epsilon}(y_i^3 + a_i y_i + b_i),$$
$$\frac{da_i}{dt} = \sigma\left(\frac{a_{i-1} - 2a_i + a_{i+1}}{(\Delta x)^2}\right) + b_i + \frac{0.07(y_i - 0.7)(y_i - 1.3)}{(y_i - 0.7)(y_i - 1.3) + 0.1},$$

$$\frac{db_i}{dt} = \sigma\left(\frac{b_{i-1} - 2b_i + b_{i+1}}{(\Delta x)^2}\right) + (1 - a_i^2)b_i - a_i - 0.4y_i + \frac{0.035(y_i - 0.7)(y_i - 1.3)}{(y_i - 0.7)(y_i - 1.3) + 0.1},$$

where $i = 1, 2, \ldots, N$ and $\Delta x = 1/N$. The boundary conditions are given as periodic and accordingly written as

$$\begin{aligned} y_0(t) &= y_N(t), & a_0(t) &= a_N(t), & b_{N+1}(t) &= b_1(t), \\ y_{N+1}(t) &= y_1(t), & a_{N+1}(t) &= a_1(t), & b_{N+1}(t) &= b_1(t). \end{aligned}$$

Initial conditions are given as

$$y_i(0) = 0, \quad a_i(0) = -2\cos(2i\pi\,\Delta x), \quad b_i(0) = 2\sin(2i\pi\,\Delta x), \quad i = 1, 2, \ldots, N.$$

3.6. Combustion

Combustion models can be built from advection–diffusion–reaction equations, often involving many coupled PDEs that represent interactions among multiple reacting species. These PDEs typically describe conservation of energy, conservation of mass (also known as the continuity equation), and conservation of momentum [34].

For this example, we choose a simplified combustion model [28], in which a single PDE is used to represent a combustion reaction. The flow in the domain is prescribed, rather than being influenced by reactions that occur in the simulation. This simplified model is written as

$$\frac{\partial c}{\partial t} = \left[1 + U_0 \sin\left(\frac{\pi x}{L}\right)\right]\left(\frac{\gamma}{\beta}\frac{\partial^3 c}{\partial x^2} - \frac{\partial c}{\partial x}\right) + f(c), \tag{10}$$

where $x \in [10, 50]$, c is the mass fraction of the reacted products, $f(c)$ is the reaction model, $U_0 \in [0, 1]$ is a parameter that controls density and velocity changes of the fluid domain, L is the length scale of the domain, and the parameters γ and β are representative of the average diffusivity. Following the numerical experiments of [28], the model has been evaluated for $U_0 = \{0, 0.75, 0.99\}$ and for the Fisher–Kolmogorov–Petrovskii–Piskunov (FKPP), ignition, and general order-m Fisher reaction models. The three reaction models are written, respectively, as

$$\begin{aligned} f_{\text{FKPP}}(c) &= \alpha_1 c(1 - c), \\ f_{\text{ignition}}(c) &= \alpha_2(1 - c)(c - c_s)^+, \\ f_{\text{Fisher}}(c) &= \alpha_3 c^m(1 - c). \end{aligned}$$

where α_1, α_2, and α_3 represent rates of reaction, c_s is a reaction threshold, $(\cdot)^+ = \max(0, \cdot)$, and m is the general Fisher non-linearity [28]. Values α_2 and α_3 are calculated from the relation $\max(f(c)) = \frac{\alpha_1}{4}$ given by [28], such that all three reaction types are of approximately the same magnitude. The calculations are as follows:

$$\begin{aligned} f'_{\text{ignition}}(c) = 0 &\implies c = \frac{c_s + 1}{2} &&\implies \alpha_2 = \frac{\alpha_1}{(1 - c_s)^2}; \\ f'_{\text{Fisher}}(c) = 0 &\implies c = \frac{m}{m + 1} &&\implies \alpha_3 = \frac{\alpha_1}{4(\frac{m}{m+1})^m(1 - \frac{m}{m+1})}. \end{aligned}$$

Boundary conditions are periodic. The initial condition of the system is given as

$$c(x, 0) = \exp\left[-\frac{(x - x_0)^2}{\sigma_0}\right].$$

This problem is uniformly discretized using N grid points for the unknowns and centred differences for the spatial derivatives. Therefore, the combustion model is written as

$$\begin{aligned} \frac{dc_i}{dt} = &\left[1 + U_0 \sin\left(\frac{i\pi\,\Delta x}{L}\right)\right]\left[\frac{\gamma}{\beta(\Delta x)^2}(c_{i-1} + c_i + c_{i+1}) + \frac{1}{2\Delta x}(c_{i+1} - c_{i-1})\right] \\ &+ f(c_i), \end{aligned}$$

where $i = 1, 2, \ldots, N$, the boundary conditions are imposed as $c_0(t) = c_N(t)$ and $c_{N+1}(t) = c_1(t)$, and initial condition as $c_i(0) = \exp[-(x - x_i)^2/\sigma_0]$. This example utilizes parameters $L = 1$, $\beta = 1$, $\gamma = 0.1$, $\alpha_1 = 0.1$, $c_s = 0.6$, $m = 10$, and $\sigma_0 = 10$.

3.7. Tumour angiogenesis

The next example is a model of tumour angiogenesis [16]. It tracks the growth of a tumour in one spatial dimension. The underlying ADR equations are comprised of non-linear advection, linear diffusion, and non-linear reaction terms. Two

quantities are modelled in this domain: (i) ρ is the blood vessel density and (ii) c is the concentration of the tumour-angiogenesis factor, which stimulates the growth of blood vessels. The model is scaled to the domain $x \in [0, 1]$ and is given as a system of two PDEs,

$$\frac{\partial \rho}{\partial t} = \epsilon \frac{\partial^2 \rho}{\partial x^2} - \frac{\partial}{\partial x}\left(\kappa \frac{\partial c}{\partial x}\rho\right) + \mu\rho(1-\rho)(c-c^*)^+ - \beta\rho, \tag{11a}$$

$$\frac{\partial c}{\partial t} = \delta \frac{\partial^2 c}{\partial x^2} - \lambda c - \frac{\alpha\rho c}{\gamma + c}. \tag{11b}$$

Initial and boundary conditions for ρ and c are given as

$$\rho(x,0) = \begin{cases} 0 & \text{if } 0 \le x < 1, \\ 1 & \text{if } x = 1, \end{cases} \qquad c(x,0) = \cos\left(\frac{1}{2}\pi x\right),$$

$$\rho(0,t) = 0, \quad \rho(1,t) = 1, \qquad c(0,t) = 1, \quad c(1,t) = 0.$$

This PDE model can be transformed into an IVP by first applying the product rule to the second term of the RHS of (11a) and second, by discretizing the spatial derivatives of (11a) and (11b) using centred finite differences. The spatial domain is comprised of N subintervals; therefore, $\Delta x = \frac{1}{N+1}$. This spatial discretization method is known not to be optimal for this problem; however, it is sufficient to compare ARK methods on the model.

The discretized form of (11) can now be written as

$$\begin{aligned}\frac{d\rho_i}{dt} &= \frac{\epsilon}{(\Delta x)^2}(\rho_{i-1} - 2\rho_i + \rho_{i+1}) - \frac{\kappa}{(\Delta x)^2}(c_{i-1} - 2c_i + c_{i+1})\rho_i \\ &\quad - \frac{\kappa}{(\Delta x/2)^2}(\rho_{i+\frac{1}{2}}(c_{i+1} - c_i) - \rho_{i-\frac{1}{2}}(c_i - c_{i-1})) + \mu\rho_i(1-\rho_i)(c_i - c^*)^+ - \beta\rho_i, \\ \frac{dc_i}{dt} &= \frac{\delta}{(\Delta x)^2}(c_{i-1} - 2c_i + c_{i+1}) - \lambda c_i - \frac{\alpha\rho_i c_i}{\gamma + c_i},\end{aligned}$$

where $i = 1, 2, \ldots, N$ and points at half-indices are the average value between two cells, e.g., $\rho_{i+\frac{1}{2}} = (\rho_{i+1} + \rho_i)/2$. The discretized initial and boundary conditions are written

$$\rho_{0,i} = 0 \qquad \rho_0(t) = 0, \qquad \rho_{N+1}(t) = 1,$$

$$c_{0,i} = \cos\left(\frac{1}{2}\pi i \Delta x\right), \qquad c_0(t) = 1, \qquad c_{N+1}(t) = 0.$$

This example utilizes parameters $\epsilon = 10^{-3}$, $\delta = \{1, 10^{-3}\}$, $\alpha = 10$, $\beta = 4$, $\gamma = 1$, $\kappa = 0.75$, $\lambda = 1$, $\mu = 100$, $c^* = 0.2$.

3.8. Concrete rewetting

The process of concrete rewetting occurs when a hydration front passing through dry concrete, possibly as a result of rainfall or water runoff, causing unreacted particles of cement to react [35]; this reaction causes the concrete to become stronger. Chapwanya et al. [29] proposed a mathematical model of concrete rewetting, i.e., a model describing the physical and chemical structure of dry concrete that has been exposed to a boundary of new moisture. The model is a system of coupled PDEs, comprised of contributing factors from advection, diffusion, and reaction. The PDEs have one spatial dimension: a simplification of a column of concrete with an insulated or sink boundary on the top and a hydrated boundary on the bottom. The model applies to the movement of (i) isopropanol, a non-reactive solution that diffuses without altering the porosity of the concrete, and (ii) water, a solution that reacts with cement. This study considers only case (ii).

The equations for the rewetting model are

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial x}\left[D(\theta, \epsilon)\frac{\partial \theta}{\partial x}\right] - \nu(\theta - \theta_r)^+ \frac{m_w r_{\text{csh}}}{\rho_w m_{\text{csh}}}, \tag{12a}$$

$$\frac{\partial(\theta C_\alpha)}{\partial t} = \frac{\partial}{\partial x}\left(\theta D_\alpha \frac{\partial C_\alpha}{\partial x}\right) - \frac{\partial(uC_\alpha)}{\partial x} - (\theta - \theta_r)^+ r_\alpha, \tag{12b}$$

$$\frac{\partial(\theta C_\beta)}{\partial t} = \frac{\partial}{\partial x}\left(\theta D_\beta \frac{\partial C_\beta}{\partial x}\right) - \frac{\partial(uC_\beta)}{\partial x} - (\theta - \theta_r)^+ r_\beta, \tag{12c}$$

$$\frac{\partial(\theta C_q)}{\partial t} = \frac{\partial}{\partial x}\left(\theta D_q \frac{\partial C_q}{\partial x}\right) - \frac{\partial(uC_q)}{\partial x} + (\theta - \theta_r)^+ (r_{\text{csh}} - k_{\text{prec}} C_q + k_{\text{diss}} C_g), \tag{12d}$$

$$\frac{\partial(\theta C_g)}{\partial t} = (\theta - \theta_r)^+ (k_{\text{prec}} C_q - k_{\text{diss}} C_g), \tag{12e}$$

where $(\theta - \theta_r)^+ = \max(0, \theta - \theta_r)$; θ is the volumetric water content; C_α, C_β, C_q, and C_g are the respective constituent concentrations of C_3S in concrete, C_2S in concrete, calcium-silicate hydrate (C–S–H) in liquid, and solid C–S–H gel; r_α, r_β,

and r_{csh} are functions that govern the rates of reaction; D is the effective diffusivity; D_α, D_β, and D_q are the respective diffusivities of C_3S in concrete, C_2S in concrete, and C–S–H in liquid. The remaining variables are substance-specific constants defined in [29].

The domain is one-dimensional with length L. It is discretized into N cells of equal width $\Delta x = \frac{L}{N}$ using standard finite volumes. The spatially discretized form is written as

$$\begin{aligned}
\frac{d\theta_i}{dt} &= \frac{1}{(\Delta x)^2}\left[D_{i+\frac{1}{2}}\left(\theta_{i+1}-\theta_i\right) - D_{i-\frac{1}{2}}\left(\theta_i-\theta_{i-1}\right)\right] - \nu(\theta_i-\theta_r)^+\frac{m_w r_{\text{csh}}}{\rho_w m_{\text{csh}}},\\
\frac{d(\theta_i C_{\alpha,i})}{dt} &= \frac{D_\alpha}{(\Delta x)^2}\left[\theta_{i+\frac{1}{2}}\left(C_{\alpha,i+1}-C_{\alpha,i}\right) - \theta_{i-\frac{1}{2}}\left(C_{\alpha,i}-C_{\alpha,i-1}\right)\right]\\
&\quad - \frac{u_{i+\frac{1}{2}}C_{\alpha,i+\frac{1}{2}} - u_{i-\frac{1}{2}}C_{\alpha,i-\frac{1}{2}}}{\Delta x} - (\theta_i-\theta_r)^+ r_\alpha,\\
\frac{d(\theta_i C_{\beta,i})}{dt} &= \frac{D_\beta}{(\Delta x)^2}\left[\theta_{i+\frac{1}{2}}\left(C_{\beta,i+1}-C_{\beta,i}\right) - \theta_{i-\frac{1}{2}}\left(C_{\beta,i}-C_{\beta,i-1}\right)\right]\\
&\quad - \frac{u_{i+\frac{1}{2}}C_{\beta,i+\frac{1}{2}} - u_{i-\frac{1}{2}}C_{\beta,i-\frac{1}{2}}}{\Delta x} - (\theta_i-\theta_r)^+ r_\beta,\\
\frac{d(\theta_i C_{q,i})}{dt} &= \frac{D_q}{(\Delta x)^2}\left[\theta_{i+\frac{1}{2}}\left(C_{q,i+1}-C_{q,i}\right) - \theta_{i-\frac{1}{2}}\left(C_{q,i}-C_{q,i-1}\right)\right]\\
&\quad - \frac{u_{i+\frac{1}{2}}C_{q,i+\frac{1}{2}} - u_{i-\frac{1}{2}}C_{q,i-\frac{1}{2}}}{\Delta x} + (\theta_i-\theta_r)^+(r_{\text{csh}} - k_{\text{prec}}C_{q,i} + k_{\text{diss}}C_{g,i}),\\
\frac{d(\theta_i C_{g,i})}{dt} &= (\theta_i-\theta_r)^+(k_{\text{prec}}C_{q,i} - k_{\text{diss}}C_{g,i}),
\end{aligned}$$

where $u_{i+\frac{1}{2}} = -D_{i+\frac{1}{2}}(\theta_{i+1}-\theta_i)/\Delta x$ and $D_i = D(\theta_i, \epsilon_i)$. Points at half-indices are calculated as the average between cells, e.g., $\theta_{i+\frac{1}{2}} = (\theta_i + \theta_{i+1})/2$.

The initial conditions are given as

$$\theta(x,0) = \theta_{\min}, \qquad C_\alpha(x,0) = C_\alpha^0, \qquad C_\beta(x,0) = C_\beta^0,$$
$$C_q(x,0) = 0, \qquad C_g(x,0) = 0,$$

where

$$C_j^0 = (1-f_j)\omega_j C_{mix},$$
$$C_{mix} = \frac{\rho_{cem}}{R_{w/c}\left(\rho_{cem}/\rho_w\right) + R_{a/c}\left(\rho_{cem}/\rho_{agg}\right) + 1},$$

and ρ_{cem} is the original cement density, ρ_{agg} is the particle density of aggregates, $R_{w/c}$ and $R_{a/c}$ are initial water-to-cement and aggregate-to-cement ratios by mass, ω_j is the mass fraction for each constitute $j = \alpha, \beta$, and $f_j \in [0, 1]$ is the fractional degree of hydration of each constituent at the end of the curing stages.

Two cases for boundary conditions are considered. In both cases, the water content is subject to a sink boundary condition at $x = 0$, and all solution components are subject to insulating boundary conditions at $x = L$. The first case treats the boundary at $x = 0$ as insulated for all constituent concentrations, and thus the full set of conditions is written as

$$\theta(0,t) = \theta_{\max} - \frac{C_g(0,t)}{\rho_g}, \qquad \frac{dC_j}{dx}(0,t) = 0,$$
$$\frac{d\theta}{dx}(L,t) = 0, \qquad \frac{dC_j}{dx}(L,t) = 0,$$

for $j = \alpha, \beta, q$ and where ρ_g is the C–S–H gel density. For brevity, this case is referred to as having an insulated boundary. The second case replaces the insulating boundary condition on the constituent concentrations at $x = 0$ with the sink boundary condition $C_j(0,t) = 0, j = \alpha, \beta, q$. For brevity, this case is referred to as having a sink boundary.

4. Performance results and discussion of numerical experiments

This section gives the performance results of the IMEX and RKC methods applied to the eight problems in the test suite with physics-based splitting and dynamic linearization in terms of work-precision diagrams. The CPU times reported are the minimum of three runs, and the accuracies reported are the 2-norms of the errors at the end of the simulations,

$$\|\mathbf{y}(t_{\text{final}}) - \mathbf{y}_{n_{\text{steps}}}\|_2,$$

where $\mathbf{y}_{n_{\text{steps}}}$ is the numerical solution of a given integration method for a given problem at t_{final}.

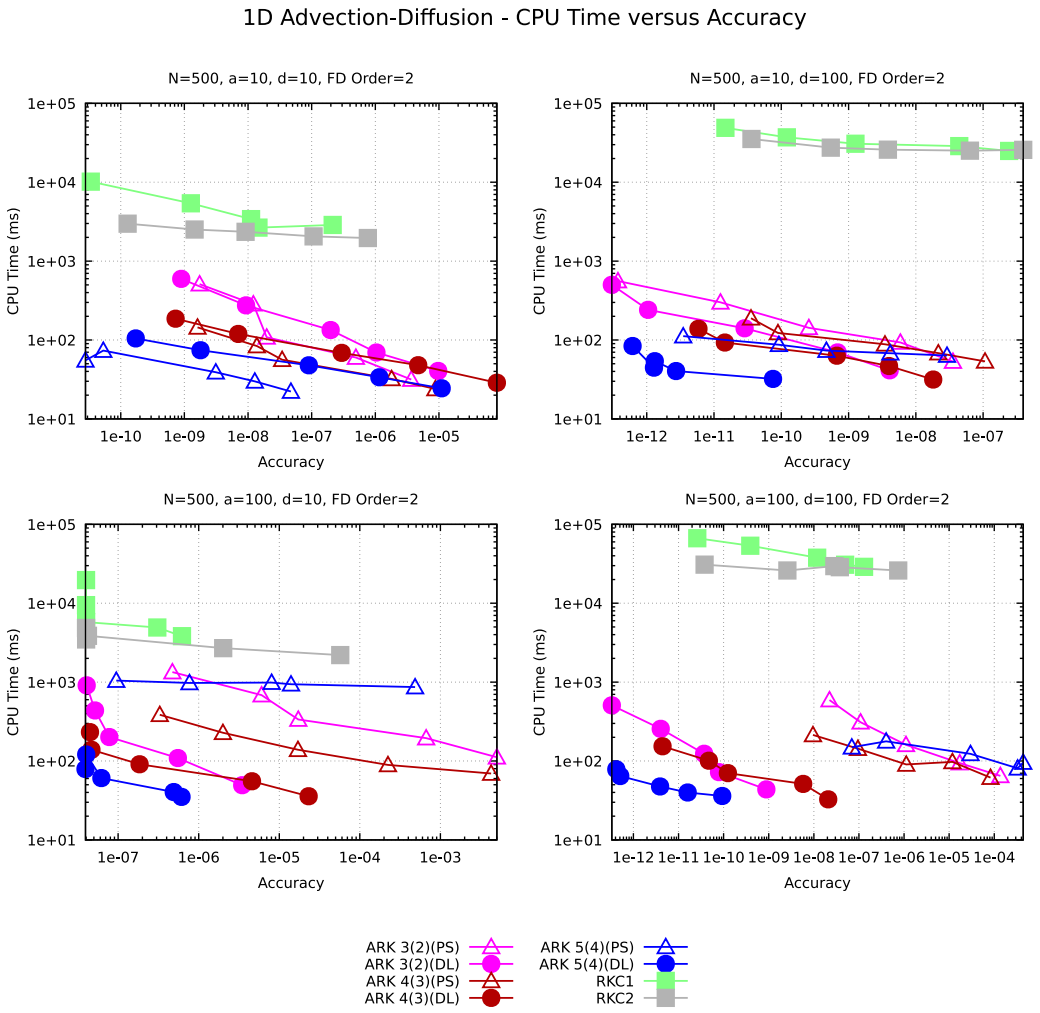


Fig. 1. CPU time versus accuracy of IMEX and RKC methods applied to a one-dimensional advection-diffusion equation. Tolerances range from 1e−4 to 1e−8.

4.1. 1D advection-diffusion

The one-dimensional advection-diffusion model described in Section 3.1 is evaluated for varying levels of diffusivity and strength of the advection term using $N = 500$. The final simulation time is $t_{\text{final}} = 0.1$. The work-precision plots are shown in Fig. 1, from which it can be seen that dynamic linearization generally outperforms physics-based splitting at a given order, especially for strict tolerances and as the strength of the advection term increases. These observations support that dynamic linearization outperforms physics-based splitting because the latter treats the advection term explicitly and thus faces issues of numerical stability. The RKC methods severely underperform by two or three orders of magnitude. Perhaps at first counter-intuitively, the performance if the RKC methods deteriorates as the size of the diffusion term increases. This result, however, may be less surprising given that the stability region of the RKC methods is optimized for pure diffusion — any presence of advection would increasingly undermine the stability of RKC methods as diffusion increased. The model was also run using fourth- and sixth-order centred finite difference methods but yielded no significant change to the results.

4.2. 2D heat transfer

The two-dimensional heat transfer model described in Section 3.2 is evaluated on four sets of grids: 70×10, 140×20, 210×30, and 280×40, and the final simulation time $t_{\text{final}} = 50$. The work-precision plots for each of the cases are shown in Fig. 2. In general, dynamic linearization outperforms physics-based splitting, with the performance being approximately equal for the ARK3(2) method. Unlike the previous one-dimensional example, the RKC methods are now more competitive

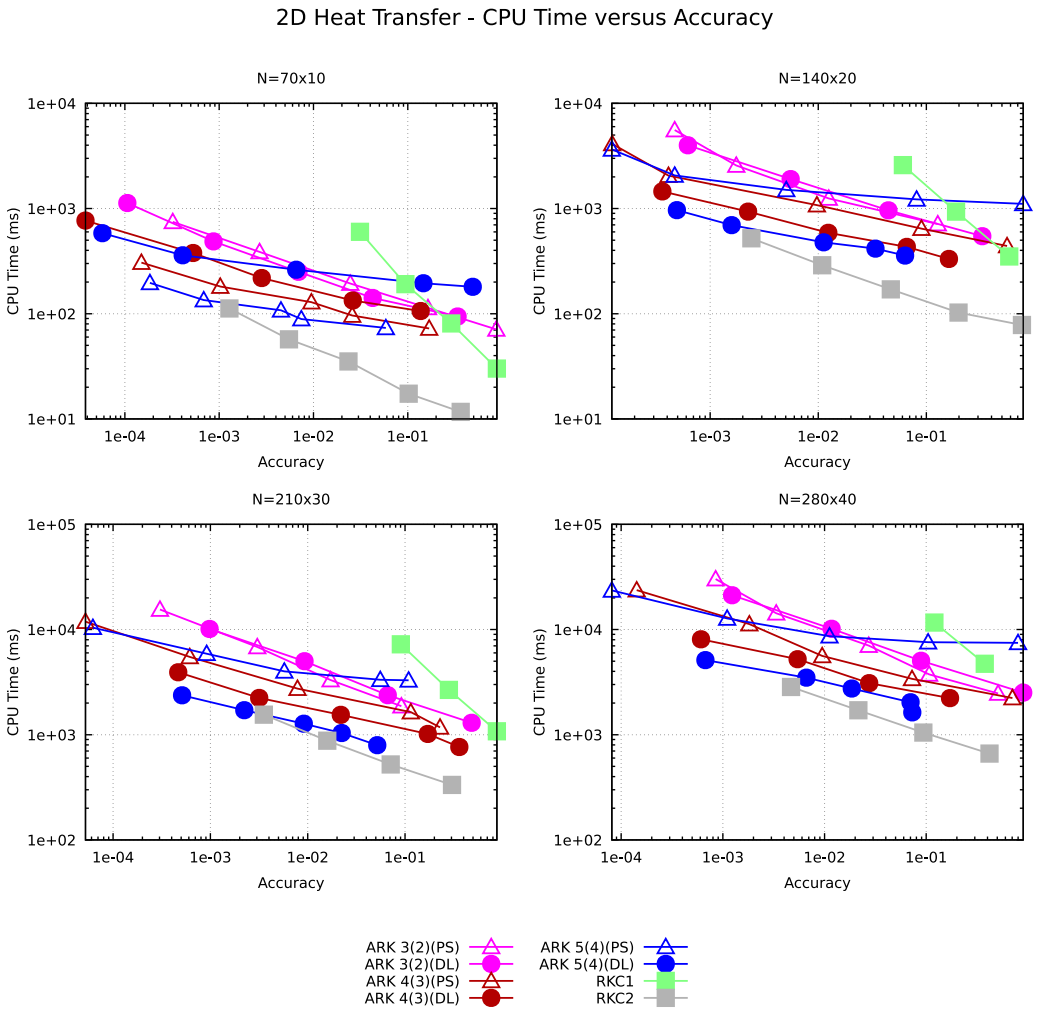


Fig. 2. CPU time versus accuracy of IMEX and RKC methods applied to the two-dimensional heat transfer problem. Tolerances range from 1e−4 to 1e−9.

with the IMEX methods. RKC2 is able to solve the problem faster than the IMEX methods for accuracies of around 1e−3 and coarser. This outperformance of RKC2 is due to the increase in linear algebra cost of the implicit solve in the IMEX methods. However, the advantages offered by higher-order methods dominate for stricter accuracies.

The problem was also run using second- and third-order backward differences for first derivatives, and fourth- and sixth-order central differences for second derivatives. The use of these methods did not yield a significant change to the results.

4.3. 1D brusselator

The one-dimensional Brusselator problem described in Section 3.3 is solved for $N = 500, 1000$. The work-precision plots are given in Fig. 3.

The results show that the optimal type of splitting is highly dependent on the order of the IMEX method. For the ARK5(4) method and $\alpha = 0.02$, dynamic linearization generally takes fewer steps and less CPU time than physics-based splitting and is thus more efficient. Conversely, when $\alpha = 0.002$, the ARK3(2) method using dynamic linearization is outperformed by physics-based splitting. In the other cases, there is no discernible difference between the performance of physics-based splitting and dynamic linearization. The RKC methods underperform in all cases except for the coarsest tolerances.

4.4. 2D brusselator

The two-dimensional Brusselator problem is run for $N = 50, 75$. The work-precision plots are given in Fig. 4. The results for the two-dimensional Brusselator problem have some similarities with the one-dimensional case; however,

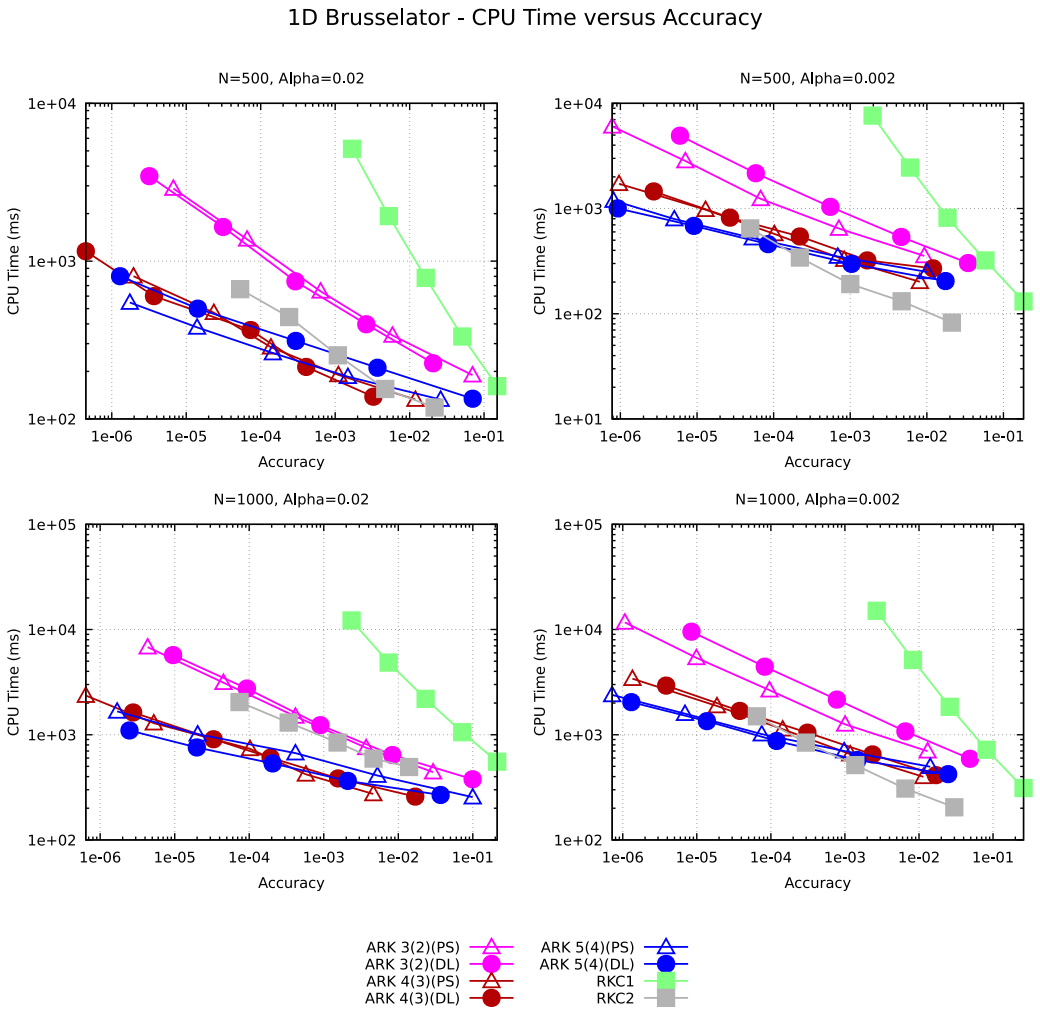


Fig. 3. CPU time versus accuracy of IMEX and RKC methods applied to a one-dimensional Brusselator problem. Tolerances range from 1e−4 to 1e−8.

the outperformance of the ARK3(2) physics-based splitting vanishes. These results demonstrate how even though the reaction term of the Brusselator problem is relatively non-stiff, it is still beneficial to use dynamic linearization. Unlike the one-dimensional case, the RKC2 method now outperforms all of the IMEX methods; however, it cannot achieve the highest levels of accuracy. RKC1 underperforms relative to the IMEX methods.

Both Brusselator problems were also run with fourth- and sixth-order finite difference methods but yielded no significant change to the results.

4.5. CUSP

The CUSP problem described in Section 3.5 is run with $N = 32$. The final simulation time $t_f = 1.1$. The work-precision plot is given in Fig. 5.

The results show that the IMEX methods that use dynamic linearization outperform all the IMEX methods using physics-based splitting by an order of magnitude. RKC1 can outperform the IMEX methods tested by as much as an order of magnitude, however, only for the coarsest tolerances.

4.6. Combustion

The combustion problem described in Section 3.6 was evaluated for the FKPP, ignition, and Fisher reaction models for a grid size $N = 1600$ and $U_0 = \{0, 0.75, 0.99\}$. The work-precision plots are given in Fig. 6.

In every case, IMEX methods of orders four and five using dynamic linearization outperform the same methods using physics-based splitting. As U_0 increases, the relative performance of dynamic linearization also appears to increase, to the

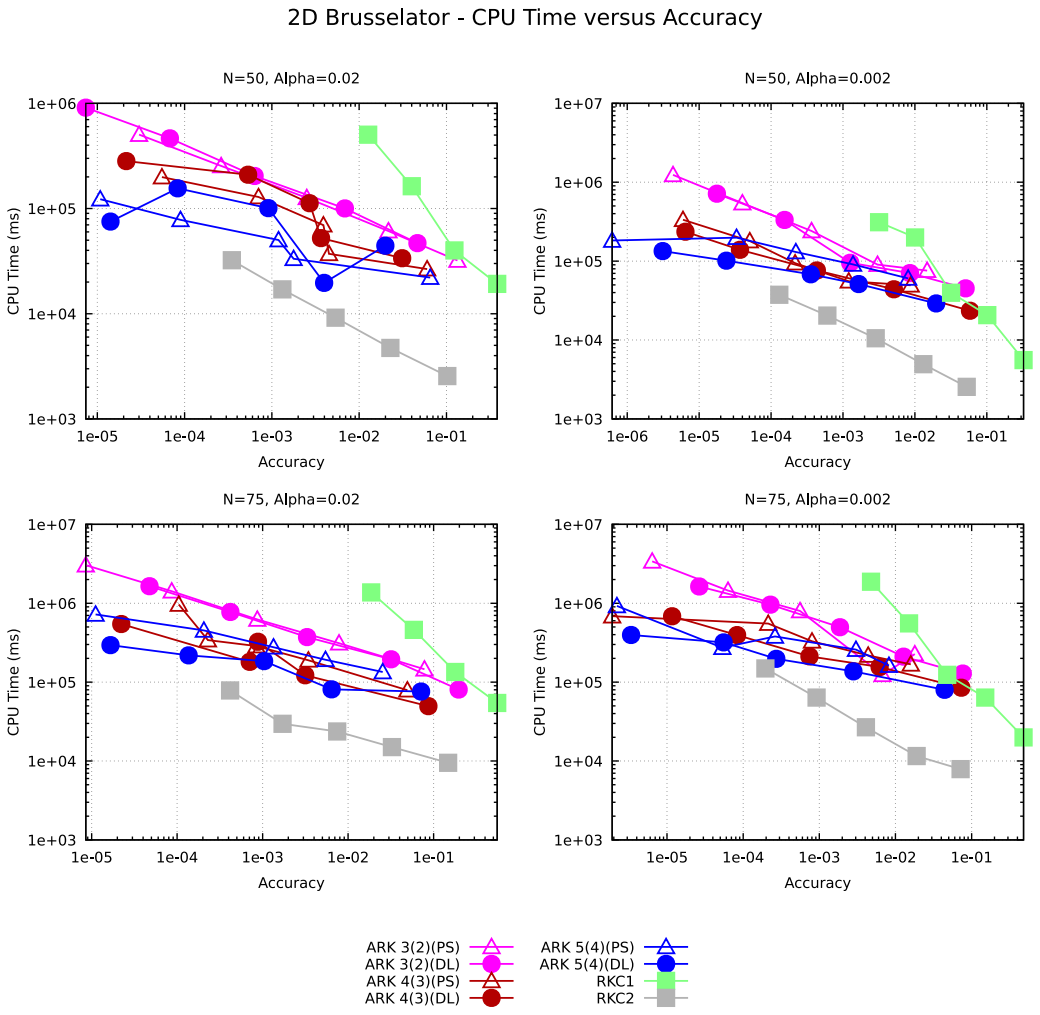


Fig. 4. CPU time versus accuracy of IMEX and RKC methods applied to a two-dimensional Brusselator problem. Tolerances range from 1e−4 to 1e−8.

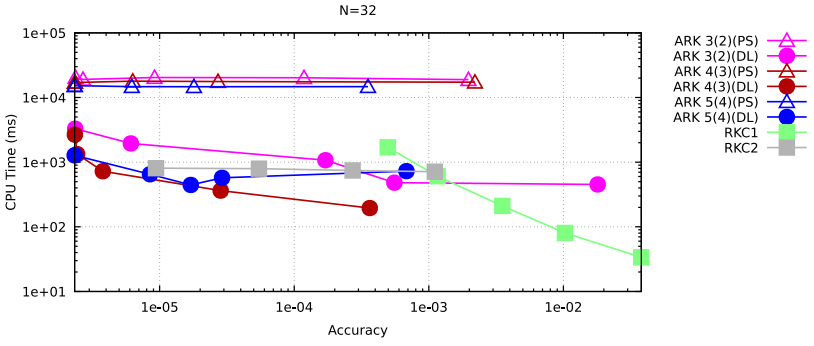


Fig. 5. CPU time versus accuracy of IMEX and RKC methods applied to the CUSP problem. Tolerances range from 1e−4 to 1e−8.

point that all methods using dynamic linearization outperform those that do not. When the combustion model involves the FKPP or Fisher reactions, the relative outperformance of dynamic linearization is also more significant than for the ignition reaction. The ignition problem nonetheless demonstrates the superior performance of dynamic linearization. In all cases, the RKC methods of orders one and two are outperformed by the IMEX methods.

The model was also run with higher-order spatial discretizations but yielded no significant change to the results.

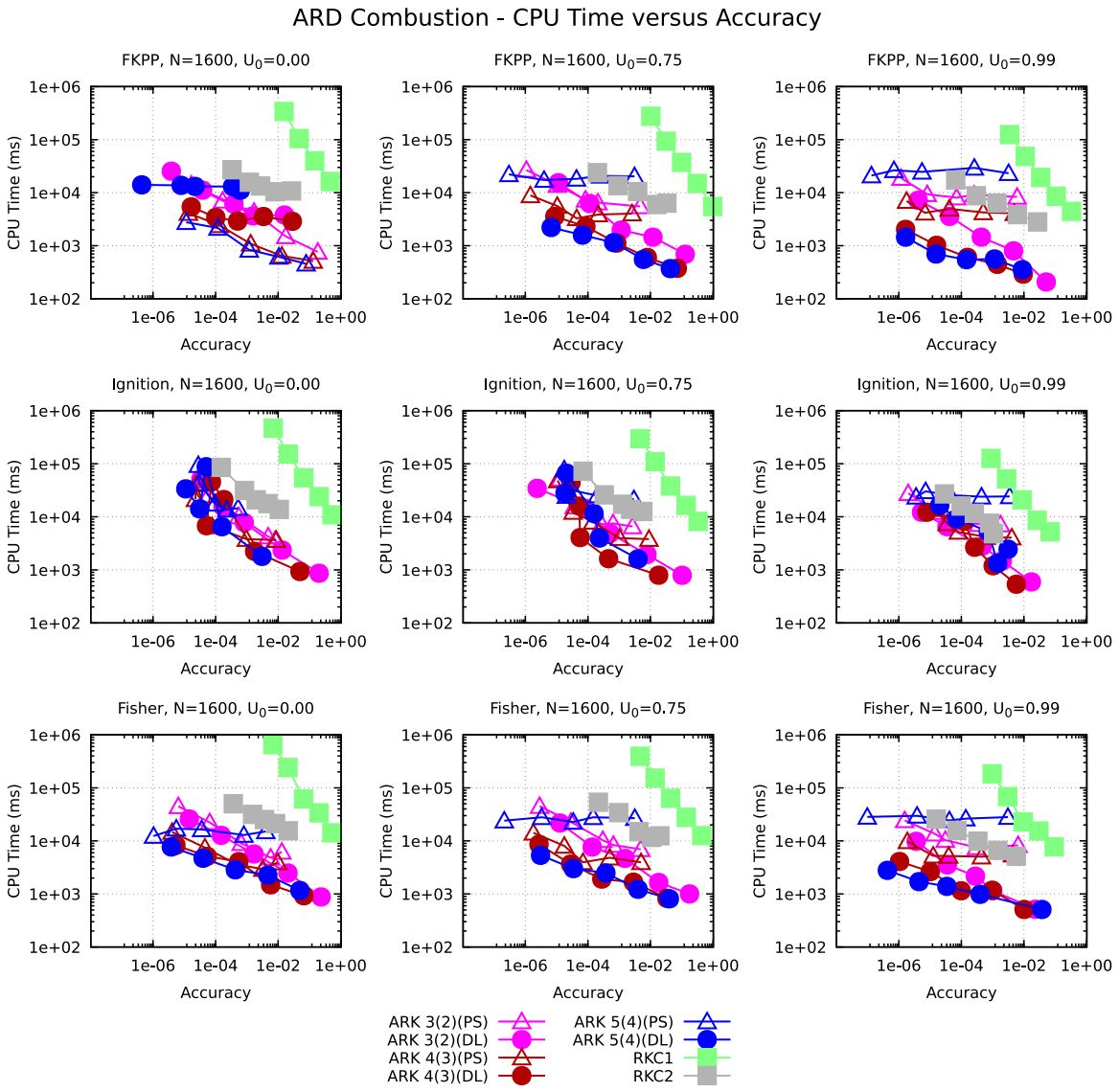


Fig. 6. CPU time versus accuracy of IMEX and RKC methods applied to the three combustion models. Tolerances range from 1e−4 to 1e−9.

4.7. Tumour angiogenesis

The tumour-angiogenesis problem described in Section 3.7 is run for $N = \{500, 1000\}$ and diffusion coefficients $\delta = \{1, 0.001\}$. The work-precision plots are shown in Fig. 7. In general, results show methods using dynamic linearization perform comparably to physics-based splitting on these problems. However, the tumour-angiogenesis model yields the noteworthy result that at strict tolerances, IMEX methods using physics-based splitting are unable to complete the simulations. The recorded errors for methods using physics-based splitting either show that a minimum stepsize of 10^{-12} was reached or that a maximum number of steps (1,000,000) was reached. The methods using dynamic linearization successfully solve the problem at all requested tolerances. These results suggest that physics-based splitting does not appropriately capture the stiffness of this problem, i.e., all of the stiffness is not captured in the diffusion term. In contrast, dynamic linearization seems to be able to adapt to the changing relative contributions of the stiffness over time.

4.8. Concrete rewetting

The concrete-rewetting problem, described in Section 3.8, is run for $N = \{100, 200\}$, for both the insulated and sink boundaries. The work-precision plots are shown in Fig. 8.

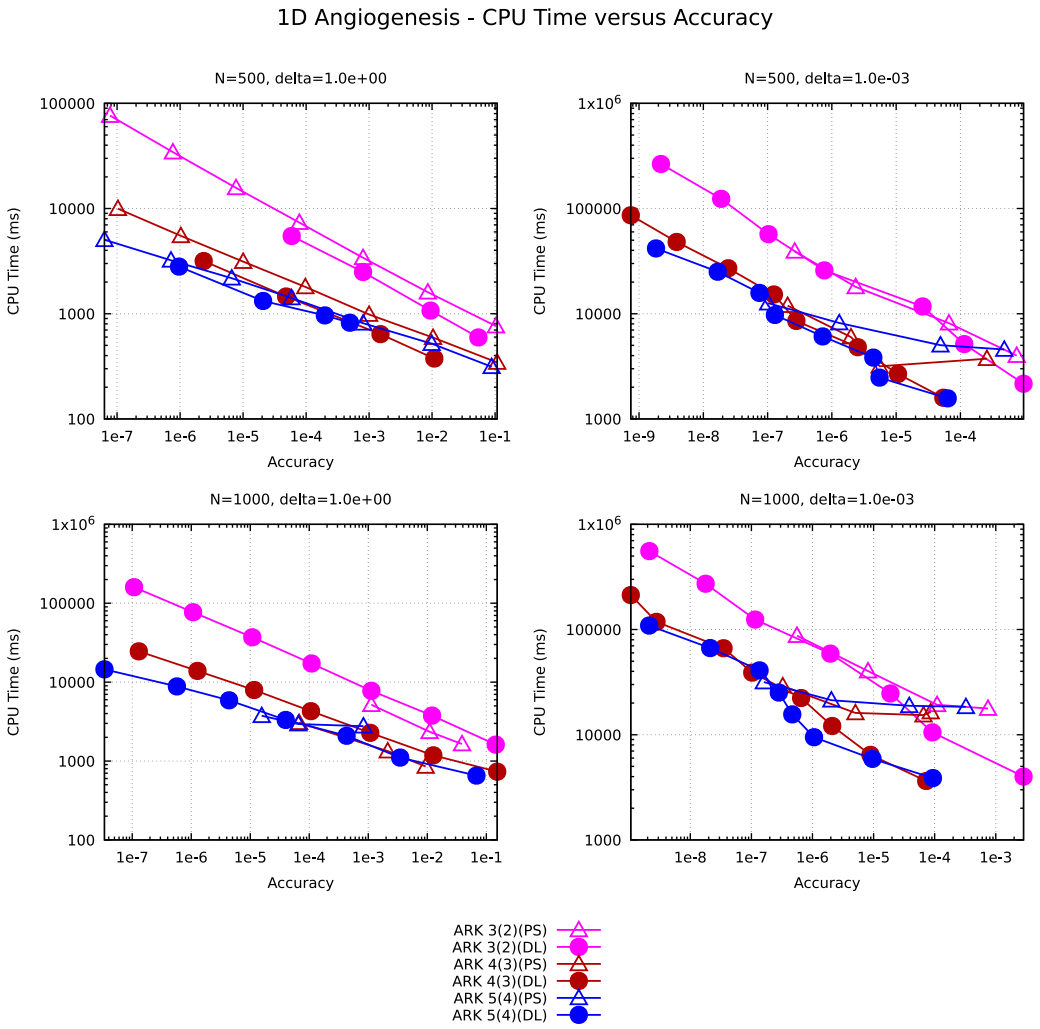


Fig. 7. CPU time versus accuracy of IMEX methods applied to a one-dimensional tumour-angiogenesis problem. Tolerances range from 1e−5 to 1e−12.

For this example, the RKC1 and RKC2 methods substantially outperform all of the IMEX methods, regardless of the type of splitting used. This is because the process of concrete rewetting is diffusion dominated, i.e., the size of the dominant eigenvalues of the Jacobian of the right-hand side in both cases of the sink and insulated boundaries are located on the negative real axis, playing to the strength of the stability region encompassed by the RKC methods.

For the ARK3(2) and ARK4(3) methods, dynamic linearization typically outperforms physics-based splitting, and the lower-order methods outperform ARK5(4). For the ARK5(4) method, dynamic linearization typically outperforms physics-based splitting, with the exception of $N = 200$ with an insulated boundary, in which dynamic linearization is both slower and fails for a tolerance of 10^{-6}. There is still no ARK5(4) method generally accepted as being sufficiently robust [4], and indeed this example demonstrates some of its shortcomings.

5. Conclusions

This study compares the performance of IMEX-RK methods with physics-based splitting to dynamic linearization on advection–diffusion–reaction PDEs by applying both types of splittings on a suite of eight test problems: 1D Advection-Diffusion, 2D Heat Transfer, 1D and 2D Brusselator, CUSP, combustion, tumour angiogenesis, and concrete rewetting. RKC methods are also applied to the test problems as a baseline for comparison with classical monolithic methods.

The results show that dynamic linearization generally outperforms physics-based splitting on the 1D advection–diffusion, CUSP, combustion, tumour-angiogenesis, and concrete-rewetting problems. The outperformance is often on the order of an order of magnitude or more. Furthermore, dynamic linearization succeeds at solving the tumour-angiogenesis problem at strict tolerances where physics-based splitting fails. On the Brusselator problems, the performance of the two

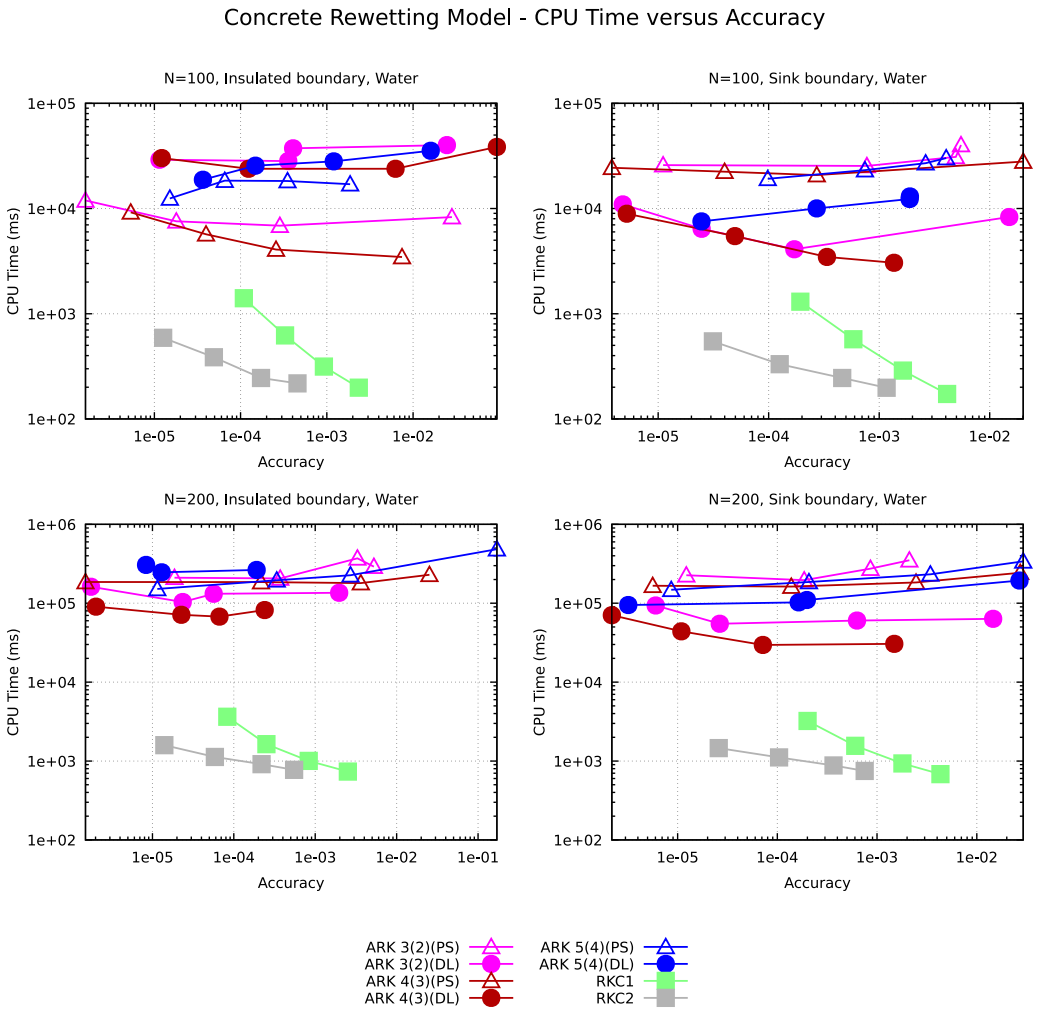


Fig. 8. CPU time versus accuracy of IMEX and RKC methods applied to the concrete-rewetting problem. Tolerances range from 1e−4 to 1e−7.

splitting methods is comparable, with the edge leaning to dynamic linearization as the problem size (dimension, number of grid points) increases.

In general, the RKC methods are outperformed by the IMEX methods. However, in the 2D heat transfer, Brusselator, and concrete rewetting problems, they are able to outperform the IMEX methods at coarse tolerances. For these problems, RKC methods can be the best performers by one or two orders of magnitude. In other cases, however, they may underperform to the point of not being infeasible. The RKC methods used are generally unable to produce solutions at fine tolerances.

These results indicate that dynamic linearization is a promising splitting method for solving ADR IVPs. Although it does not outperform the traditional physics-based splitting in all cases, it does outperform on the vast majority of the cases tested. The cases in which dynamic linearization succeeds where physics-based splitting fails particularly support the use of dynamic linearization for ADR PDEs as a worthy splitting strategy.

The problems within the test suite used in this study were chosen to represent a diverse set of ADR PDEs. However, they are by no means comprehensive. Future studies on the use of dynamic linearization for ADR PDEs may help to strengthen the understanding for the situations (e.g., relative stiffness of physical processes, order and stability of underlying RK methods, accuracy required, etc.) in which dynamic linearization performs well and why. The development and investigation of 3-additive methods, e.g., those that treat each of the advection–reaction–diffusion terms separately in a physics-based splitting, are reported elsewhere.

Acknowledgement

This work was supported by the Natural Sciences and Engineering Research Council of Canada under its Discovery Grants program via grant RGPN-2020-04467.

References

[1] G.J. Cooper, A. Sayfy, Additive methods for the numerical solution of ordinary differential equations, Math. Comp. 35 (152) (1980) 1159–1172, URL: http://dx.doi.org/10.2307/2006380, http://dx.doi.org/10.2307/2006380.

[2] G.J. Cooper, A. Sayfy, Additive Runge-Kutta methods for stiff ordinary differential equations, Math. Comp. 40 (161) (1983) 207–218, URL: http://dx.doi.org/10.2307/2007370, http://dx.doi.org/10.2307/2007370.

[3] C.A. Kennedy, M.H. Carpenter, Additive Runge-Kutta schemes for convection-diffusion-reaction equations, Appl. Numer. Math. 44 (2003) 139–181.

[4] C.A. Kennedy, M.H. Carpenter, Higher-order additive runge–kutta schemes for ordinary differential equations, Appl. Numer. Math. 136 (2019) 183–205.

[5] Z. Jackiewicz, General Linear Methods for Ordinary Differential Equations, John Wiley & Sons, Inc., Hoboken, NJ, 2009, URL: http://dx.doi.org/10.1002/9780470522165 http://dx.doi.org/10.1002/9780470522165.

[6] R.I. McLachlan, G.R.W. Quispel, Splitting methods, Acta Numer. 11 (2002) 341–434, URL: http://dx.doi.org/10.1017/S0962492902000053, http://dx.doi.org/10.1017/S0962492902000053.

[7] U.M. Ascher, S. Ruuth, R.J. Spiteri, Implicit-explicit Runge-Kutta methods for time-dependent partial differential equations, Appl. Numer. Math. 25 (1997) 151–167.

[8] S. González-Pinto, D. Hernández-Abreu, M.S. Pérez-Rodríguez, A. Sarshar, S. Roberts, A. Sandu, A unified formulation of splitting-based implicit time integration schemes, J. Comput. Phys. 448 (2022) Paper No. 110766, 22, URL: http://dx.doi.org/10.1016/j.jcp.2021.110766, http://dx.doi.org/10.1016/j.jcp.2021.110766.

[9] C.J. Vogl, A. Steyer, D. Reynolds, P.A. Ullrich, C.S. Woodward, Evaluation of implicit-explicit additive Runge-Kutta integrators for the HOMME-NH dynamical core, J. Adv. Modelling Earth Syst. 11 (12) (2019) 4228–4244.

[10] D.J. Gardner, J.E. Guerra, D.R. Reynolds, P.A. Ullrich, C.S. Woodward, Implicit-explicit (IMEX) Runge-Kutta methods for non-hydrostatic atmospheric models, Geosci. Model Dev. 11 (4) (2018) 1497–1515.

[11] S. Boscarino, R. Büger, P. Mulet, G. Russo, L. Villada, On linearly implicit IMEX Runge-Kutta methods for degenerate convection-diffusion problems modeling polydisperse sedimentation, Bull. Braz. Math. Soc. (N.S.) 47 (2016) 171–185.

[12] V.T. Luan, D.L. Michels, Efficient exponential time integration for simulating nonlinear coupled oscillators, J. Comput. Appl. Math. 391 (2021).

[13] J.D. Lawson, Generalized Runge-Kutta processes for stable systems with large Lipschitz constants, SIAM J. Numer. Anal. 4 (1967) 372–380, URL: http://dx.doi.org/10.1137/0704033, http://dx.doi.org/10.1137/0704033.

[14] V.T. Luan, A. Ostermann, Exponential rosenbrock methods of order five - construction, analysis and numerical comparisons, J. Comput. Appl. Math. 255 (2014) 417–431.

[15] K. Pieper, C.K. Sockwell, M. Gunzburger, Exponential time differencing for mimetic multilayer ocean models, J. Comput. Phys. 398 (2019).

[16] W. Hundsdorfer, J.G. Verwer, Numerical solution of time-dependent advection-diffusion-reaction equations, in: Springer Series in Computational Mathematics, vol. 33, Springer-Verlag, Berlin, 2003.

[17] E. Hairer, S.P. Nørsett, G. Wanner, Solving ordinary differential equations I: Nonstiff problems, in: Springer Series in Computational Mathematics, vol. 8, Springer-Verlag, Berlin, 1993.

[18] L.F. Shampine, B.P. Sommeijer, J.G. Verwer, IRKC: An IMEX solver for stiff diffusion-reaction PDEs, J. Comput. Appl. Math. 196 (2) (2006) 485–497.

[19] C.J. Zbinden, Partitioned Runge-Kutta-Chebyshev methods for diffusion-advection-reaction problems, SIAM J. Sci. Comput. 33 (4) (2011) 1707–1725.

[20] E. Celledoni, B.K. Kometa, Semi-Lagrangian Runge–Kutta exponential integrators for convection dominated problems, J. Sci. Comput. 41 (1) (2009) 139–164.

[21] J.G. Verwer, W. Hundsdorfer, B.P. Sommeijer, Convergence properties of the Runge-Kutta-Chebyshev method, Numer. Math. 57 (1) (1990) 157–178.

[22] A. Preuss, `pythODE++`, 2021, https://github.com/uofs-simlab/pythODEpp.

[23] A. Kroshko, Integrating-Factor-Based 2-Additive Runge-Kutta Methods for Advection-Reaction-Diffusion Equations (Master's thesis), Department of Computer Science, University of Saskatchewan, 2011.

[24] G. Satir, D. Brown, C++ - The Core Language: A Foundation for C Programmers, O'Reilly, 1995.

[25] F. Mazzia, C. Magherini, Test Set for Initial Value Problem Solvers, Release 2.4, Technical Report 4, Department of Mathematics, University of Bari, Italy, 2008.

[26] W.H. Enright, J.D. Pryce, Two FORTRAN packages for assessing initial value methods, ACM Trans. Math. Software 13 (1) (1987) 1–27.

[27] E. Hairer, G. Wanner, Solving ordinary differential equations II: Stiff and differential-algebraic problems, in: Springer Series in Computational Mathematics, vol. 14, Springer-Verlag, Berlin, 1996.

[28] F. Bianco, S. Chibbaro, R. Prud'homme, Étude d'une équation de convection-réaction-diffusion en écoulement compressible, 2011, Presented at: Rencontre du non linéaire.

[29] M. Chapwanya, W. Liu, J.M. Stockie, A model for reactive porous transport during re-wetting of hardened concrete, J. Eng. Math. 65 (2009) 53–73.

[30] A. Griewank, D. Juedes, J. Utke, Algorithm 755: ADOL-C: A package for the automatic differentiation of algorithms written in C/C++, ACM Trans. Math. Software 22 (2) (1996) 131–167, URL: http://dx.doi.org/10.1145/229473.229474, http://dx.doi.org/10.1145/229473.229474.

[31] T.A. Davis, Suitesparse, 2021, https://github.com/DrTimothyAldenDavis/SuiteSparse/releases.

[32] T.A. Davis, Algorithm 832: UMFPACK V4.3-an unsymmetric-pattern multifrontal method, ACM Trans. Math. Softw. 30 (2) (2004) 196–199, URL: http://dx.doi.org/10.1145/992200.992206, http://dx.doi.org/10.1145/992200.992206.

[33] L.M. Jiji, Heat Convection, second ed., Springer Berlin Heidelberg, 2009.

[34] T. Echekki, E. Mastorakos, Turbulent combustion modeling, in: Fluid Mechanics and its Applications, vol. 95, 95, Springer Netherlands, 2011.

[35] C. Hall, Anomalous diffusion in unsaturated flow: fact or fiction? Cem. Concr. Res. 37 (3) (2007) 378–385.